

10. Validation trong Spring Boot

10.1. Giới thiệu về Validation

Validation (kiểm tra hợp lệ) là một phần quan trọng trong việc phát triển ứng dụng, đảm bảo rằng dữ liệu đầu vào từ người dùng hoặc các hệ thống khác tuân thủ các quy tắc nghiệp vụ và định dạng mong muốn. Việc kiểm tra dữ liệu ngay từ đầu giúp ngăn chặn các lỗi tiềm ẩn, bảo vệ tính toàn vẹn của dữ liệu và cải thiện trải nghiệm người dùng bằng cách cung cấp phản hồi rõ ràng về các lỗi nhập liệu.

Spring Boot tích hợp chặt chẽ với Bean Validation API (JSR 380), một chuẩn Java cho việc định nghĩa và thực thi các ràng buộc validation. Hibernate Validator là một triển khai phổ biến của Bean Validation API.

10.2. Thêm Dependency

Để sử dụng Bean Validation trong Spring Boot, bạn cần thêm dependency `spring-boot-starter-validation` :

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Dependency này sẽ tự động kéo về Hibernate Validator và các thư viện cần thiết khác.

10.3. Các Annotation Validation phổ biến

Bean Validation cung cấp một tập hợp các annotation để định nghĩa các ràng buộc trên các trường của lớp POJO. Dưới đây là một số annotation phổ biến:

10.3.1. Ràng buộc chung

- `@NotNull` : Trường không được null.
- `@NotEmpty` : Trường không được null và không rỗng (áp dụng cho String, Collection, Map, Array).
- `@NotBlank` : Trường không được null, không rỗng và không chỉ chứa khoảng trắng (áp dụng cho String).
- `@Size(min=x, max=y)` : Kích thước của trường phải nằm trong khoảng min và max (áp dụng cho String, Collection, Map, Array).
- `@Min(value)` : Giá trị số phải lớn hơn hoặc bằng `value` .
- `@Max(value)` : Giá trị số phải nhỏ hơn hoặc bằng `value` .
- `@DecimalMin(value)` : Giá trị số phải lớn hơn hoặc bằng `value` (áp dụng cho BigDecimal, BigInteger, String).
- `@DecimalMax(value)` : Giá trị số phải nhỏ hơn hoặc bằng `value` (áp dụng cho BigDecimal, BigInteger, String).
- `@Positive` : Giá trị số phải dương.
- `@PositiveOrZero` : Giá trị số phải dương hoặc bằng 0.
- `@Negative` : Giá trị số phải âm.
- `@NegativeOrZero` : Giá trị số phải âm hoặc bằng 0.

10.3.2. Ràng buộc cho chuỗi

- `@Email` : Chuỗi phải là một địa chỉ email hợp lệ.
- `@Pattern(regex="regex")` : Chuỗi phải khớp với biểu thức chính quy (regular expression).

10.3.3. Ràng buộc cho ngày tháng

- `@Past` : Ngày phải ở trong quá khứ.
- `@PastOrPresent` : Ngày phải ở trong quá khứ hoặc hiện tại.

- `@Future` : Ngày phải ở trong tương lai.
- `@FutureOrPresent` : Ngày phải ở trong tương lai hoặc hiện tại.

10.4. Áp dụng Validation trong Spring Boot

10.4.1. Validation trên Request Body (REST APIs)

Để kích hoạt validation cho các đối tượng được gửi trong request body (ví dụ: JSON), bạn sử dụng annotation `@Valid` hoặc `@Validated` trên tham số của phương thức controller.

Ví dụ:

Lớp DTO/Model với Validation Annotations:

Java

```
import javax.validation.constraints.*;
import java.math.BigDecimal;

public class ProductRequest {

    @NotBlank(message = "Product name is required")
    @Size(min = 3, max = 100, message = "Product name must be between 3 and 100 characters")
    private String name;

    @Size(max = 500, message = "Description cannot exceed 500 characters")
    private String description;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be positive")
    private BigDecimal price;

    @NotBlank(message = "Category is required")
    private String category;

    @Email(message = "Invalid email format")
    private String contactEmail;

    // Getters and Setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
```

```

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }

    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }

    public String getContactEmail() { return contactEmail; }
    public void setContactEmail(String contactEmail) { this.contactEmail =
contactEmail; }
}

```

Controller:

Java

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import javax.validation.Valid;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    @PostMapping
    public ResponseEntity<String> createProduct(@Valid @RequestBody
ProductRequest productRequest) {
        // Nếu validation thành công, logic nghiệp vụ sẽ được thực thi
        System.out.println("Product received: " + productRequest.getName());
        return new ResponseEntity<>("Product created successfully!",
HttpStatus.CREATED);
    }

    @PutMapping("/{id}")
    public ResponseEntity<String> updateProduct(@PathVariable Long id, @Valid
@RequestBody ProductRequest productRequest) {
        System.out.println("Product updated: " + productRequest.getName());
        return ResponseEntity.ok("Product updated successfully!");
    }
}

```

Khi một request đến `createProduct` hoặc `updateProduct` với dữ liệu không hợp lệ, Spring sẽ tự động ném ra `MethodArgumentNotValidException`. Bạn có thể xử lý ngoại lệ này bằng `@ControllerAdvice` như đã trình bày trong tài liệu "05. Handle Exception".

10.4.2. Validation trên Path Variables và Request Parameters

Để validate các tham số đường dẫn (`@PathVariable`) và tham số request (`@RequestParam`), bạn cần thêm `@Validated` vào lớp controller và đặt các annotation validation trực tiếp trên các tham số.

Ví dụ:

Java

```
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

import javax.validation.constraints.Min;
import javax.validation.constraints.Max;
import javax.validation.constraints.Pattern;

@RestController
@RequestMapping("/api/users")
@Validated // Kích hoạt validation cho các tham số của phương thức
public class UserController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getUserById(
        @PathVariable @Min(value = 1, message = "User ID must be
positive") Long id) {
        System.out.println("Fetching user with ID: " + id);
        return ResponseEntity.ok("User details for ID: " + id);
    }

    @GetMapping("/search")
    public ResponseEntity<String> searchUsers(
        @RequestParam @NotBlank(message = "Keyword cannot be empty")
String keyword,
        @RequestParam(defaultValue = "0") @Min(value = 0, message = "Page
number must be non-negative") int page,
        @RequestParam(defaultValue = "10") @Max(value = 50, message =
"Page size cannot exceed 50") int size) {
        System.out.println("Searching users with keyword: " + keyword + ",
```

```

page: " + page + ", size: " + size);
    return ResponseEntity.ok("Search results for: " + keyword);
}

@GetMapping("/email")
public ResponseEntity<String> getUserByEmail(
    @RequestParam @Email(message = "Invalid email format") String
email) {
    System.out.println("Fetching user by email: " + email);
    return ResponseEntity.ok("User details for email: " + email);
}
}

```

Khi validation thất bại cho `@PathVariable` hoặc `@RequestParam`, Spring sẽ ném ra `ConstraintViolationException`. Bạn cũng cần xử lý ngoại lệ này trong `GlobalExceptionHandler`.

Cập nhật `GlobalExceptionHandler` để xử lý `ConstraintViolationException`:

Java

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

import javax.validation.ConstraintViolation;
import javax.validation.ConstraintViolationException;
import java.util.List;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // ... (các handler khác)

    @ExceptionHandler({ConstraintViolationException.class})
    public ResponseEntity<ErrorResponse>
    handleConstraintViolationException(ConstraintViolationException ex,
    WebRequest request) {
        List<String> errors = ex.getConstraintViolations().stream()
            .map(ConstraintViolation::getMessage)
            .collect(Collectors.toList());

        ErrorResponse errorResponse = new ErrorResponse(

```

```

        HttpStatus.BAD_REQUEST.value(),
        HttpStatus.BAD_REQUEST.getReasonPhrase(),
        "Validation failed",
        request.getDescription(false),
        errors
    );
    return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
}
}

```

10.4.3. Custom Validation Annotations

Bạn có thể tạo các annotation validation tùy chỉnh để áp dụng các quy tắc nghiệp vụ phức tạp hơn.

Ví dụ: `@ValidProductName`

Annotation Definition:

Java

```

import javax.validation.Constraint;
import javax.validation.Payload;
import java.lang.annotation.*;

@Documented
@Constraint(validatedBy = ProductNameValidator.class)
@Target({ElementType.FIELD, ElementType.METHOD, ElementType.PARAMETER,
        ElementType.ANNOTATION_TYPE})
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidProductName {
    String message() default "Invalid product name";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

```

Validator Implementation:

Java

```

import javax.validation.ConstraintValidator;
import javax.validation.ConstraintValidatorContext;

public class ProductNameValidator implements

```

```

ConstraintValidator<ValidProductName, String> {

    @Override
    public void initialize(ValidProductName constraintAnnotation) {
        // Có thể khởi tạo logic tại đây nếu cần
    }

    @Override
    public boolean isValid(String productName, ConstraintValidatorContext
context) {
        if (productName == null || productName.trim().isEmpty()) {
            return false; // @NotBlank hoặc @NotEmpty nên xử lý trường hợp
này
        }
        // Ví dụ: Tên sản phẩm không được chứa ký tự đặc biệt và phải bắt đầu
bằng chữ cái
        return productName.matches("[a-zA-Z][a-zA-Z0-9 ]*$");
    }
}

```

Sử dụng Custom Annotation:

Java

```

public class ProductRequest {

    @ValidProductName(message = "Product name must start with a letter and
contain only letters, numbers, and spaces")
    private String name;

    // ...
}

```

10.5. Validation Groups

Validation Groups cho phép bạn nhóm các ràng buộc validation và chỉ áp dụng chúng trong các kịch bản cụ thể (ví dụ: một tập hợp các ràng buộc cho thao tác tạo, và một tập hợp khác cho thao tác cập nhật).

Ví dụ:

Define Groups:

Java

```
public interface OnCreate {}  
public interface OnUpdate {}
```

Apply Groups to Annotations:

Java

```
import javax.validation.constraints.*;  
import java.math.BigDecimal;  
  
public class ProductRequest {  
  
    @Null(groups = OnCreate.class, message = "ID must be null for creation")  
    @NotNull(groups = OnUpdate.class, message = "ID must not be null for  
update")  
    private Long id;  
  
    @NotBlank(groups = {OnCreate.class, OnUpdate.class}, message = "Product  
name is required")  
    private String name;  
  
    @NotNull(groups = OnCreate.class, message = "Price is required for  
creation")  
    @Positive(groups = OnCreate.class, message = "Price must be positive for  
creation")  
    private BigDecimal price;  
  
    // Getters and Setters  
}
```

Validate with Groups in Controller:

Java

```
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.validation.annotation.Validated;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping("/api/products")  
public class ProductController {
```

```

@PostMapping
public ResponseEntity<String> createProduct(@Validated(OnCreate.class)
@RequestBody ProductRequest productRequest) {
    System.out.println("Creating product: " + productRequest.getName());
    return new ResponseEntity<>("Product created successfully!",
HttpStatus.CREATED);
}

@PutMapping
public ResponseEntity<String> updateProduct(@Validated(OnUpdate.class)
@RequestBody ProductRequest productRequest) {
    System.out.println("Updating product: " + productRequest.getName());
    return ResponseEntity.ok("Product updated successfully!");
}
}

```

10.6. Bản chất của Validation trong Spring Boot

Bản chất của Validation trong Spring Boot là cung cấp một cơ chế mạnh mẽ, linh hoạt và tích hợp sâu sắc để đảm bảo tính hợp lệ của dữ liệu đầu vào. Bằng cách tận dụng Bean Validation API và Hibernate Validator, Spring Boot cho phép các nhà phát triển định nghĩa các quy tắc validation một cách khai báo (declarative) bằng cách sử dụng các annotation trực quan. Điều này giúp tách biệt logic validation khỏi logic nghiệp vụ, làm cho mã nguồn sạch hơn, dễ đọc và dễ bảo trì hơn. Khả năng xử lý lỗi validation một cách tập trung thông qua `@ControllerAdvice` và tùy chỉnh các thông báo lỗi giúp cải thiện đáng kể trải nghiệm người dùng và độ tin cậy của ứng dụng. Validation là một yếu tố không thể thiếu để xây dựng các ứng dụng web an toàn và mạnh mẽ.